



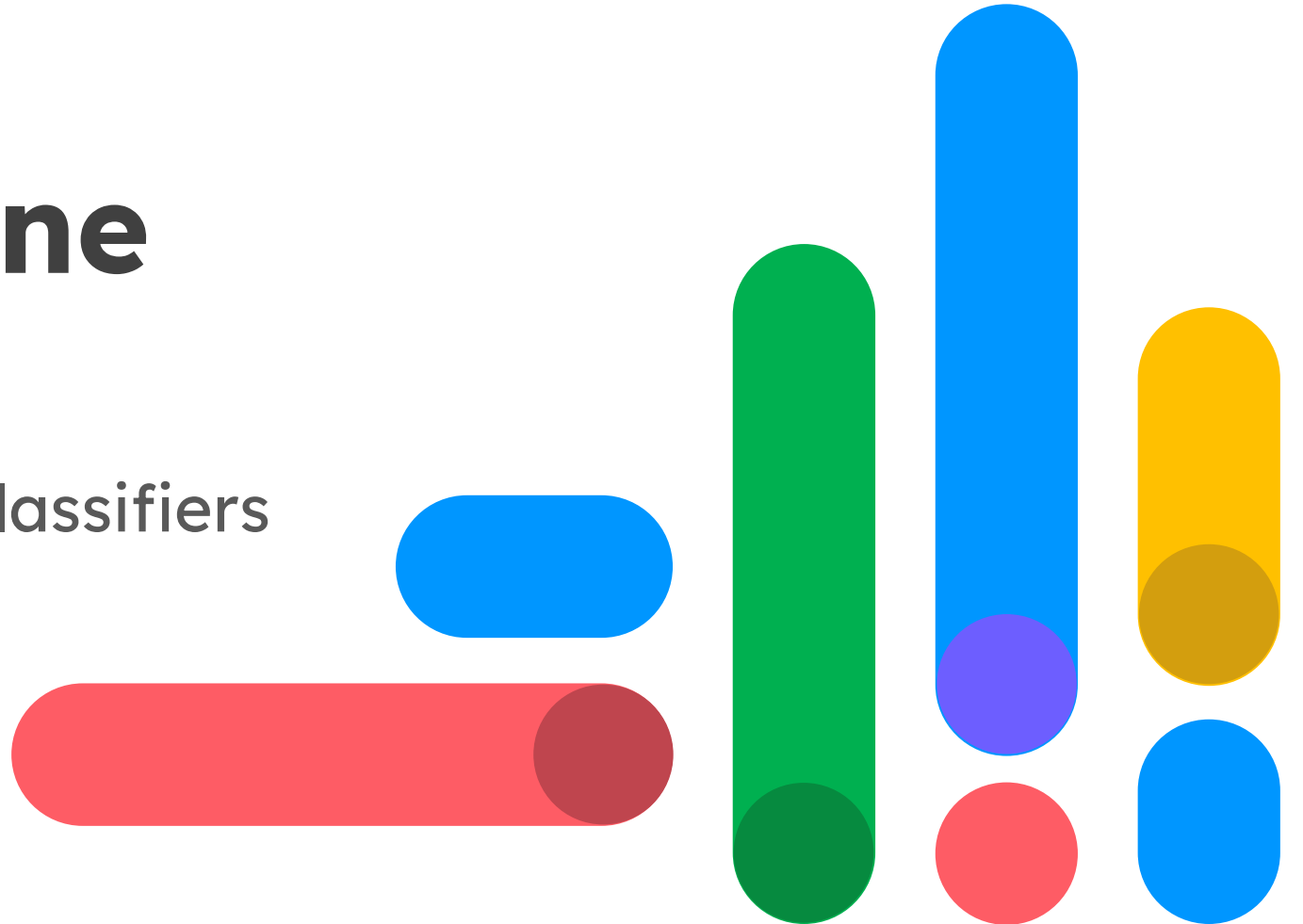
Conventional ML Methods

Applied Machine Learning

Classifier Models :: Modern Classifiers

Assistant Professor | **Chun-Hsiang Chan**

Department of Geography
National Taiwan Normal University



Outline

#GeoAI #Mobility #Geography #STDS

- 01** Logistic Regression
- 02** K-Nearest Neighbors
- 03** Naïve Bayes
- 04** Decision Tree
- 05** Bagging Methods
- 06** Random Forest
- 07** ExtraTree

- 08** Boosting Methods
- 09** AdaBoost
- 10** Gradient Boosting
- 11** XGBoost
- 12** LightGBM
- 13** CatBoost
- 14** Support Vector Machine

XGBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

From Gradient Boosting to XGBoost, ...

We have learned that Gradient Boosting sequentially adds new trees to reduce the current loss.

XGBoost keeps exactly this boosting idea, but makes the optimization of each new tree more explicit.

It uses not only the gradient but also second-order information, introduces model complexity directly into the objective function, and uses this objective to determine the prediction of each leaf and whether a split is worthwhile.

XGBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

From Gradient Boosting to XGBoost, ...

Gradient Boosting

- Add a new weak learner sequentially
- Fit the **negative gradient** of the current loss
- Each new tree corrects the mistakes made so far
- Flexible: works with many differentiable loss functions

XGBoost

- Explicitly controls **model complexity**
- Optimizes the **leaf weights analytically**
- Uses an explicit criterion to decide whether a split is worthwhile
- Implements computational optimizations for large-scale learning

XGBoost keeps the boosting philosophy, but makes each tree addition more explicitly optimized and regularized.

XGBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

What is the next step of XGBoost (XGB), comparing to Gradient Boosting (GB)?

GB tells directions only

- XGB further consider the curvature of loss which may accelerate to find the optima.

XGB adopts two elements:

$$g_i = \frac{\partial L}{\partial \hat{y}_i} \Rightarrow \text{gradient} \Rightarrow \text{direction}$$

$$h_i = \frac{\partial^2 L}{\partial \hat{y}_i^2} \Rightarrow \text{Hessian} \Rightarrow \text{curvature}$$

Model complexity is neglected in GB

- Basically, boosting could avoid overfitting via tree depth, learning rate, number of trees, and subsampling.
 - In XGB, it directly consider model complexity as a variable in the objective function.
- ➔ Does the improvement justify the additional complexity?

XGBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

The key function of XGBoost is

$$\mathcal{L}^{(t)} = \underbrace{\sum_{i=1}^n l(y_i, \hat{y}_i^{(t)})}_{\text{The prediction loss}} + \underbrace{\sum_{k=1}^t \Omega(f_k)}_{\text{Model complexity}}$$

The first term is similar to Gradient Boosting, and the second term is the penalty of model complexity.

$$\hat{y}_i^{(t)} = \underbrace{\hat{y}_i^{(t-1)}}_{\text{We have t-1 tree}} + \underbrace{f_t(x_i)}_{\text{We want find a new tree } f_t}$$

XGBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Here, we introduce the **regularization of XGBoost** for the penalty of **model complexity**.

$$\Omega(f) = \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2$$

T : number of leaves

γ : Penalty for adding a leaf

w_j : Prediction score of leaf

λ : L2 regularization on leaf weights

This equation tells us that a good tree has to a large loss reduction with less model complexity.

XGBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

We have a problem here. At t -th iteration, what kinds of tree should we add in?

$$\hat{y}_i^{(t)} = \hat{y}_i^{(t-1)} + f_t(x_i)$$

Thus, the objective function of XGBoost will be

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l\left(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)\right) + \Omega(f_t)$$

However, the previous trees are fixed, so all we have to do is optimize the f_t . But, it is hard to optimize with this loss function. Therefore, XGBoost conduct second-order Taylor approximation.

XGBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Second-order Taylor approximation

$$l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i))$$

For the prediction of $\hat{y}_i^{(t-1)}$, we may use second-order Taylor approximation to obtain

$$l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) \approx l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i)$$
$$g_i = \frac{\partial l(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i}, h_i = \frac{\partial^2 l(y_i, \hat{y}_i^{(t-1)})}{\partial \hat{y}_i^2}$$

Thus,

$$\mathcal{L}^{(t)} \approx \sum_i \left[l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_t^2(x_i) \right] + \Omega(f_t)$$

XGBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Second-order Taylor approximation

$$\mathcal{L}^{(t)} \approx \sum_i \left[l(y_i, \hat{y}_i^{(t-1)}) + g_i f_t(x_i) + \frac{1}{2} h_i f_i^2(x_i) \right] + \Omega(f_t)$$

This is previous iteration,
just like a constant here.

Therefore, our objective function for optimization will be

$$\tilde{\mathcal{L}}^{(t)} = \sum_i \left[g_i f_t(x_i) + \frac{1}{2} h_i f_i^2(x_i) \right] + \Omega(f_t)$$

First-order Direction Second-order Curvature

XGBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Every observation will be one of leaf in the decision tree. We assume that $q(x_i) = j$ to represent that observation i is classified into leaf j .

The prediction of leaf j is w_j ; therefore, we may present as:

$$f_t(x_i) = w_{q(x_i)}$$

XGBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Now, we group all observations as leaf.

$$I_j = \{i: q(x_i) = j\}$$

This means all observation grouped in leaf j .

$$\begin{aligned}\tilde{\mathcal{L}}^{(t)} &= \sum_i \left[g_i f_t(x_i) + \frac{1}{2} h_i f_i^2(x_i) \right] \\ &= \sum_{j=1}^T \left[\left(\sum_{i \in I_j} g_i \right) w_j + \frac{1}{2} \left(\sum_{i \in I_j} h_i \right) w_j^2 \right], \text{ where } G_j = \sum_{i \in I_j} g_i \text{ and } H_j = \sum_{i \in I_j} h_i \\ \tilde{\mathcal{L}}^{(t)} &= \sum_i \left[g_i f_t(x_i) + \frac{1}{2} h_i f_i^2(x_i) \right] + \Omega(f_t) = \sum_{j=1}^T \left[G_j w_j + \frac{1}{2} H_j w_j^2 \right] + \gamma T + \frac{1}{2} \lambda \sum_{j=1}^T w_j^2 \\ &= \sum_{j=1}^T \left[G_j w_j + \frac{1}{2} (H_j + \lambda) w_j^2 \right] + \gamma T\end{aligned}$$

XGBoost

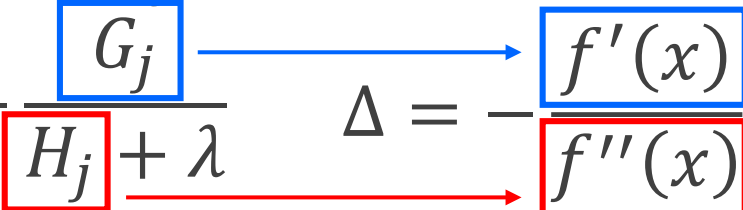
Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

What is the best prediction of each leaf?

Taking the derivative with respect to w_j ,

$$\frac{\partial \tilde{\mathcal{L}}}{\partial w_j} = G_j + (H_j + \lambda)w_j$$

$$G_j + (H_j + \lambda)w_j = 0$$

$$w_j^* = -\frac{G_j}{H_j + \lambda} \quad \Delta = -\frac{f'(x)}{f''(x)}$$


G_j, H_j represent Gradient and Curvature of loss surface, respectively.
 λ declines the speed of loss; in other words, not too aggressive.

XGBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

So, how to compute the best prediction?

$$w_j^* = -\frac{G_j}{H_j + \lambda} = \frac{\sum_{i \in I_j} (y_i - p_i)}{\sum_{i \in I_j} p_i(1 - p_i) + \lambda}$$

From **nominator**,

$$\mathcal{L} = -y \log p - (1 - y) \log(1 - p)$$
$$\because p = \sigma(F)$$

$$\because g_i = p_i - y_i \Rightarrow -g_i = y_i - p_i$$

For the **denominator**,

$$H_j = \sum_{i \in I_j} p_i(1 - p_i)$$

Ref: 05 ML Classifiers (II) pp.13

$$r_i = -\frac{\partial L_i}{\partial F_i} = -\left(\frac{\partial L_i}{\partial p_i} \frac{\partial p_i}{\partial F_i}\right)$$
$$\frac{\partial L_i}{\partial p_i} = -\frac{y_i}{p_i} + \frac{1-y_i}{1-p_i} = -\frac{y_i(1-p_i) + p_i(1-y_i)}{p_i(1-p_i)} = \frac{-y_i + y_i p_i + p_i - p_i y_i}{p_i(1-p_i)} = \frac{p_i - y_i}{p_i(1-p_i)}$$
$$\frac{\partial p_i}{\partial F_i} = p_i(1 - p_i)$$
$$\therefore r_i = -\frac{\partial L_i}{\partial F_i} = -\left(\frac{\partial L_i}{\partial p_i} \frac{\partial p_i}{\partial F_i}\right) = \frac{p_i - y_i}{p_i(1-p_i)} p_i(1 - p_i) = y_i - p_i$$

Ref: 05 ML Classifiers (II) pp.39

For the **second derivative of Bernoulli log-loss**,

$$\frac{\partial^2 L_i}{\partial F_i^2} = \frac{\partial(p_i - y_i)}{\partial F_i} = \frac{\partial p_i}{\partial F_i}, \text{ since } y_i \text{ is constant}$$

from Sigmoid derivative, $\frac{\partial p_i}{\partial F_i} = p_i(1 - p_i)$

$$\frac{\partial^2 L_i}{\partial F_i^2} = p_i(1 - p_i)$$

For a leaf,

$$L'' = \sum_{i \in I_j} p_i(1 - p_i)$$

XGBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Different from gradient boosting, XGBoost has split gain to determine whether we need a split?

$$w_j^* = -\frac{G_j}{H_j + \lambda} \Rightarrow -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T$$

$$\begin{aligned} \tilde{\mathcal{L}}^* &= \sum_{j=1}^T \left[G_j w_j^* + \frac{1}{2} (H_j + \lambda) w_j^{*2} \right] + \gamma T \\ &= \sum_{j=1}^T \left[G_j \left(-\frac{G_j}{H_j + \lambda} \right) + \frac{1}{2} (H_j + \lambda) \left(-\frac{G_j}{H_j + \lambda} \right)^2 \right] + \gamma T \\ &= \sum_{j=1}^T \left[\left(-\frac{G_j^2}{H_j + \lambda} \right) + \frac{1}{2} \left(\frac{G_j^2}{H_j + \lambda} \right) \right] + \gamma T = \sum_{j=1}^T \left[-\frac{1}{2} \left(\frac{G_j^2}{H_j + \lambda} \right) \right] + \gamma T = -\frac{1}{2} \sum_{j=1}^T \frac{G_j^2}{H_j + \lambda} + \gamma T \end{aligned}$$

XGBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Given a splitted parent node, it has a left and right node.

$$G_P = G_L + G_R$$

$$H_P = H_L + H_R$$

$$\text{Before splitting} \Rightarrow -\frac{1}{2} \frac{G_P^2}{H_P + \lambda_P}$$

$$\text{After splitting} \Rightarrow -\frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda_L} + \frac{G_R^2}{H_R + \lambda_R} \right]$$

$\therefore$ Split Gain = (children benefits – parent benefit) – cost of splitting

$$= -\frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda_L} + \frac{G_R^2}{H_R + \lambda_R} \right] - \left(-\frac{1}{2} \frac{G_P^2}{H_P + \lambda_P} \right) - \gamma$$

$$= \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda_L} + \frac{G_R^2}{H_R + \lambda_R} - \frac{G_P^2}{H_P + \lambda_P} \right] - \gamma$$

XGBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Split Gain = (children benefits – parent benefit) – cost of splitting

$$= \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda_L} + \frac{G_R^2}{H_R + \lambda_R} - \frac{G_P^2}{H_P + \lambda_P} \right] - \gamma$$

If $\text{Gain} > 0$, performing the split is advantageous. Otherwise ($\text{Gain} \leq 0$), the split should be avoided as it increases model complexity without providing sufficient benefit, despite any reduction in training error.

XGBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

To sum up, there are four crucial equations in XGBoost.

$$\mathcal{L} = \text{Loss} + \text{Regularization}$$

$$\mathcal{L}(\hat{y} + f) \approx \mathcal{L}(\hat{y}) + gf + \frac{1}{2}hf^2$$

$$w_j^* = -\frac{G_j}{H_j + \lambda}$$

$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda_L} + \frac{G_R^2}{H_R + \lambda_R} - \frac{G_P^2}{H_P + \lambda_P} \right] - \gamma$$

LightGBM

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

In practical, XGBoost is always the best and the first model performance among all ML classifiers, including the current one and the following ones.

However, Microsoft research team considered that the model training process and efficiency are not good enough; therefore, they published a modified XGBoost model, called LightGBM, designed for lightening the model training process and achieve a better or similar model performance.

So, let's take a look about LightGBM →

LightGBM

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

The core of design: XGBoost and LightGBM are both implementations of Gradient Boosted Decision Trees (GBDT). Their main difference lies in how efficiently they construct each tree. Here is the problem.

In XGBoost, it will find a tree for each iteration

$$f_t(x)$$

To minimize the loss, ...

$$\mathcal{L}^{(t)} = \sum_{i=1}^n l(y_i, \hat{y}_i^{(t-1)} + f_t(x_i)) + \Omega(f_t)$$

Via 2nd Taylor Expansion

$$\tilde{\mathcal{L}}^{(t)} = \sum_i \left[g_i f_t(x_i) + \frac{1}{2} h_i f_i^2(x_i) \right] + \Omega(f_t)$$



How can we efficiently find the best tree structure and split?



When we have a large dataset with numerous continuous values → dramatically increase the cost of searching the best split.



How to make tree construction much more cheaper?

LightGBM

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

How does LightGBM do?

For a large dataset, the features with continuous values might have several possibilities; thus, LightGBM adopts the few bins for dividing all values into few groups.

Search strategy from values to bins.

Sounds like much more efficient? Right?

But how to calculate the gradients and Hessians

LightGBM

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

For all observations in each bin, we can directly compute the sum of gradients and Hessians for split gain.

$$G_i = \sum_{i \in b} g_i; H_i = \sum_{i \in b} h_i$$

Bin	G	H
1	-5	4
2	-2	3
3	+4	5
4	+7	6

For example, $\text{Bin} \leq 2$

$$G_L = -5 - 2 = -7$$

$$H_L = 4 + 3 = 7$$

$$G_R = 11, H_R = 11$$

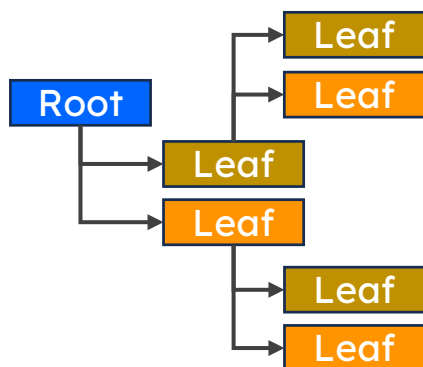
$$\text{Gain} = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{G^2}{H + \lambda} \right] - \gamma$$

LightGBM

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

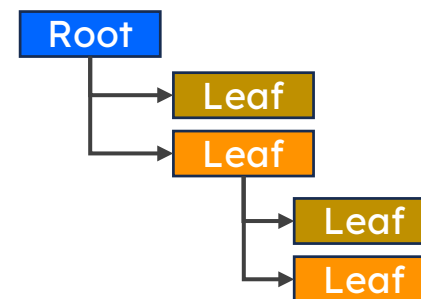
Leaf-wise Tree Growth

All typical decision tree or boosting algorithms, they usually conduct a level-wise growth.



But LightGBM does not require level-wise growth, it adopts leaf-wise growth via leaf with the largest gain.

$$\text{Choose leaf} = \underset{j}{\operatorname{argmax}} \operatorname{Gain}_j$$



LightGBM

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

You may ask why LightGBM is faster than others via leaf-wise tree growth?

If we have four leaves: L_1, L_2, L_3, L_4 , the possible gain could be $[0.3, 1.8, 0.1, 0.2]$

For level-wise growth models, they will consider multiple nodes in the same depth. However, LightGBM will choose L_2 directly, since $\max(\text{Gain})$.

Thus, leaf-wise usually reduces loss faster under the same number of leaves.

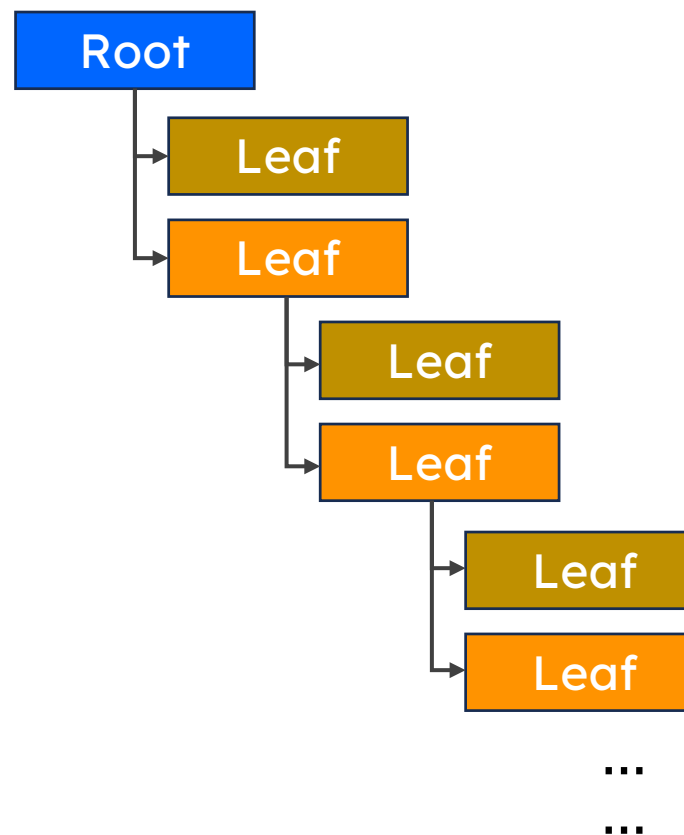
LightGBM

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

But, this approach will be prone to overfitting since the tree will be extremely unbalance, and it turns out a very deep tree under a small N .

Leaf-wise $\rightarrow$ high flexibility
 $\rightarrow$ higher overfitting risk

Therefore, both `max_depth` and `num_leaves` are vital.



LightGBM

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

The relationship between `max_depth` and `num_leaves`

If the `max_depth` of binary tree is d , then the theoretical maximum number of leaves will be 2^d .

$$L_{max} = 2^d$$

But LightGBM is leaf-wise (i.e., asymmetric tree), thus, `num_leaves` will be much more useful to control the model complexity than `max_depth`.

$$2^5 - 1 = 31$$

LightGBM

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Gradient-based One-Side Sampling (GOSS)

Given an 1M dataset, the importance of all data for the next tree is equal? **No** (said by LightGBM)

Thus, the gradient tells us:

$$g_i = \frac{\partial L}{\partial \hat{y}_i}$$

If $|g_i|$ is very high, then the model doesn't well trained for this data and the prediction error is still large.

LightGBM

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Gradient-based One-Side Sampling (GOSS)

Here, LightGBM will keep large proportion of large-gradient samples (top 20% of $|g_i|$) and randomly select some small-gradient samples.

We cannot fully withdraw all small-gradient samples; otherwise, this will cause biased data distribution.

And we will reweighting all selected all small-gradient samples → focus computation on difficult samples.

LightGBM

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Exclusive Feature Bundling (EFB)

For a high-dimensional dataset, there are some one-hot encoded features as follows.

Sample	A	B	C
1	1	0	0
2	0	1	0
3	0	0	1

We may use x_1, x_2, x_3 to represent the three items.

But, in LightGBM, it can bundle three features into a single bundled feature; thus, EFB can reduce feature dimensionality.

LightGBM

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Exclusive Feature Bundling (EFB)

Sample	A	B	C
1	1	0	0
2	0	1	0
3	0	0	1
4	1	0	0
5	0	1	0
6	0	0	1

Because all data have only one non-zero feature, it is mutually exclusive features.

1) Which features can be bundled?

$$\text{Conflict}(A, B) = \sum_i I(A_i \neq 0 \cap B_i \neq 0)$$

$$\text{Conflict}(A, B) = 0$$

$$\text{Conflict}(B, C) = 0$$

$$\text{Conflict}(A, C) = 0$$

LightGBM

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Exclusive Feature Bundling (EFB)

Sample	A	B	C
1	1	0	0
2	0	1	0
3	0	0	1
4	1	0	0
5	0	1	0
6	0	0	1

2) Histogram binning

We use a sequential number to categorized all observations within a class.

0 = inactive/default

1 = A-bin1

2 = A-bin2

0 = inactive/default

1 = B-bin1

2 = B-bin2

0 = inactive/default

1 = C-bin1

2 = C-bin2

When we see a “1” here, what does that mean?

Is A, B, or C?

LightGBM

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Exclusive Feature Bundling (EFB)

Original Feature	Original Bin	Bundled Bin
A	1	1
A	2	2
B	1	4
B	2	5
C	1	7
C	2	8

3) Add Offset

Since, A uses [0,1,2], then we set the offset of 3 for each.

A $\rightarrow$ [0, 1, 2] with $offset_A = 0$

B $\rightarrow$ [3, 4, 5] with $offset_B = 3$

C $\rightarrow$ [6, 7, 8] with $offset_C = 6$

LightGBM

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Back to the core question, **what does LightGBM lighten?**

Technique	Main idea	Benefit
Histogram	Continuous values $\rightarrow$ discrete bins	Faster split finding, lower memory
GOSS	Keep large-gradient samples	Reduce number of training samples
EFB	Bundle mutually exclusive sparse features	Reduce number of effective features
Leaf-wise growth	Split the leaf with maximum gain	Faster loss reduction, but higher overfitting risk

CatBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

CatBoost, Categorical Boosting, developed by Yandex based on Gradient Boosting algorithm.

The differences in between CatBoost and XGBoost/LightGBM are

CatBoost = Ordered Target Statistics + Ordered Boosting

Ordered target statistics and ordered boosting are mainly designed for categorical features and prediction shift caused by target leakage.

CatBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Previously, we may use one-hot encoding to vectorize the categorical data into vector space. However, if we have several categorical features or a feature with several categorical values, then that will be a problem.

Sample	City	Zip	y
A	Taipei	132	1
B	Paris	122	1
C	Tokyo	234	0
D	London	232	0
E	Tokyo	128	0
F	Paris	111	1

In LHS table, if we use one-hot encoding, zip will be high dimensional feature, then the feature space will be very sparse. Thus, CatBoost does not recommend you to do one-hot encoding in advance.

CatBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Due to high dimensionality, we may try target encoding to replace the one-hot encoding for solving this problem.

$$TE(c) = \frac{N_c(y=1)}{N_c}$$

Taipei=1; Paris=0.5

Tokyo=0.5; London=0

Sample	City	Zip	y
A	Taipei	132	1
B	Paris	122	1
C	Tokyo	234	0
D	London	232	0
E	Tokyo	128	1
F	Paris	111	0

As observed, the inclusion of location-specific features such as Taipei and London has potential target leakage, artificially inflating model metrics and leading to overly optimistic model performance.

CatBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

To solve this problem, CatBoost adopts ordered target statistics. When computing the category encoding for the i -th observation, only previous observations are used. First, we shuffle the training data by permutation.

$$\sigma(E, F, C, A, B, D)$$

Order	Sample	City	y
1	E	Tokyo	1
2	F	Paris	0
3	C	Tokyo	0
4	A	Taipei	1
5	B	Paris	1
6	D	London	0

Given $P = 0.5$; $a = 1$

$$CTR_i = \frac{\sum_{j < i} 1(x_{\sigma_j} = x_{\sigma_i}) y_{\sigma_j} + aP}{\sum_{j < i} 1(x_{\sigma_j} = x_{\sigma_i}) y_{\sigma_j} + a}$$

P is prior; a is regularization

CatBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Let's do math. Given $P = 0.5$; $a = 1$

$$CTR_i = \frac{\sum_{j < i} 1(x_{\sigma_j} = x_{\sigma_i}) y_{\sigma_j} + aP}{\sum_{j < i} 1(x_{\sigma_j} = x_{\sigma_i}) y_{\sigma_j} + a}$$

where P is prior; a is regularization

Order	Sample	City	y
1	E	Tokyo	1
2	F	Paris	0
3	C	Tokyo	0
4	A	Taipei	1
5	B	Paris	1
6	D	London	0

E	Tokyo	y=1	$CTR_E = \frac{0 + 0.5}{0 + 1} = 0.5$
F	Paris	y=0	$CTR_F = \frac{0 + 0.5}{0 + 1} = 0.5$
C	Tokyo	y=0	$CTR_C = \frac{1 + 0.5}{1 + 1} = 0.75$
A	Taipei	y=1	$CTR_A = \frac{0 + 0.5}{0 + 1} = 0.50$
B	Paris	y=1	$CTR_B = \frac{0 + 0.5}{1 + 1} = 0.25$
D	London	y=0	$CTR_D = \frac{0 + 0.5}{0 + 1} = 0.50$

CatBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Additionally, CatBoost finds that the calculating process in gradient boosting has potential prediction shift.

We use all dataset for training and calculate the gradient, which observation i is also include.

$$(x_i, y_i) \rightarrow F_{t-1} \rightarrow F_{t-1}(x_i) \rightarrow g_i$$

We can use a case for demonstration.

CatBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

In simple Gradient Boosting, assume the initial prediction is $F_0(x) = 0.5$.

The negative gradient (residual) is $r_i = y_i - F_0(x_i)$

Therefore, $r_A = 1 - 0.5 = 0.5$; $r_B = 1 - 0.5 = 0.5$; $r_C = 0 - 0.5 = -0.5$.

Sample	y
A	1
B	1
C	0

The leaf value of the first tree will be

$$\gamma = \frac{0.5 + 0.5 - 0.5}{3} = 0.167$$

Given a learning rate of 1, $F_1(x) = 0.5 + 0.167 = 0.667$

So, now the predictions of A, B, C are 0.667.

CatBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

For A, prediction of A is
 $F_1(A) = 0.667$
 $y_A = 1$

The target of A participate
the generation process of
 $F_1(A)$.
 $y_A \rightarrow F_1 \rightarrow F_1(A)$

What if A is not in the
training dataset?
 $B: y = 1; C: y = 0$

Residuals of B & C are 0.5
and -0.5 .
$$\gamma = \frac{0.5 - 0.5}{2} = 0$$

$$\therefore F_1(A_{unseen}) = 0.5 + 0 = 0.5$$

CatBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

For C, prediction of C is
 $F_1(C) = 0.667$

$$y_C = 0$$

The target of C participate
the generation process of
 $F_1(C)$.

$$y_C \rightarrow F_1 \rightarrow F_1(C)$$

What if C is not in the
training dataset?

$$A: y = 1; B: y = 0$$

Residuals of B & C are 0.5
and 0.5.

$$\gamma = \frac{0.5 + 0.5}{2} = 0.5$$

$$\therefore F_1(C_{unseen}) = 0.5 + 0.5 = 0.5$$

$$A(y=1) \rightarrow 0.5 \rightarrow 0.667$$

$$C(y=1) \rightarrow 1.0 \rightarrow 0.667$$

CatBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Ordered Boosting

Assume the permutation is $\{(x_1, y_1), (x_2, y_2), (x_3, y_3), (x_4, y_4)\}$,
When we compute the gradient of x_4 , we do not want to use the model contained (x_4, y_4) .

$F^{(3)}$ = model trained using 1,2,3

Then, we use $F^{(3)}$ to compute

$$g_4 = \frac{\partial L(y_4, F^{(3)}(x_4))}{\partial F}$$

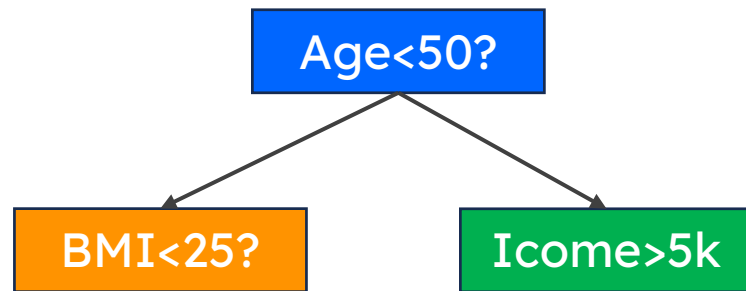
Predict sample i using a model that has not seen i

CatBoost

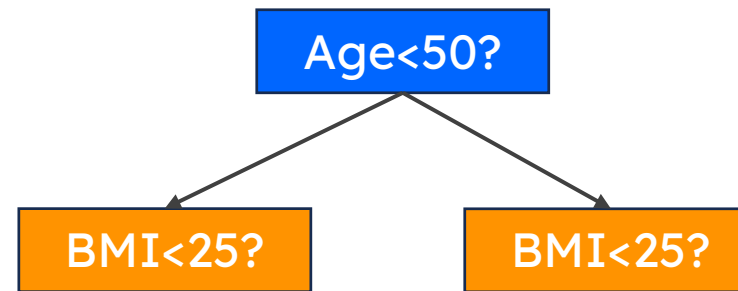
Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

One of special feature in CatBoost is symmetric trees.

Normal Decision Tree



CatBoost



This gives the tree a highly regular structure, enabling fast predictions. In CatBoost, symmetric trees serve as the default growing policy and offer significantly faster prediction times than non-symmetric trees.

CatBoost

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Here is a comparison table.

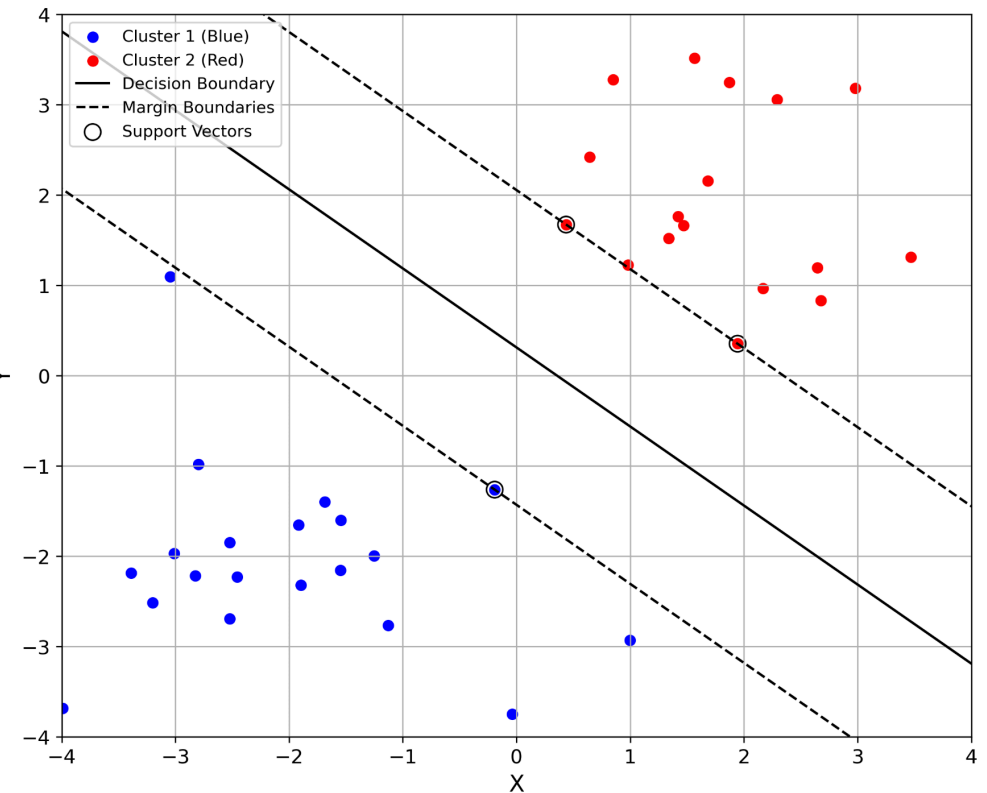
	XGBoost	LightGBM	CatBoost
Core	Regularized boosting	Efficient boosting	Reduce bias/leakage
Tree growth	Level-wise	Leaf-wise	Symmetric tree
Categorical data	Needs encoding	Built-in categorical support	Optimized
Category	External preprocessing	category-aware splits	Ordered statistics
Boosting	Standard GB	Standard GB	Ordered boosting
High cardinality category	Difficult	Good	More suitable
Training speed	High	Usually very high	Depends on Data
Prediction speed	High	High	Fast in symmetric tree

Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Different from tree-based models, the approach of **support vector machine (SVM)** is to find a hyperplane that could efficiently divide two categories.

Given a binary case, $y_i \in \{-1, +1\}$, our objective is to find a line for 2D data, a plane for 3D data, and a hyperplane for high dimensional data.



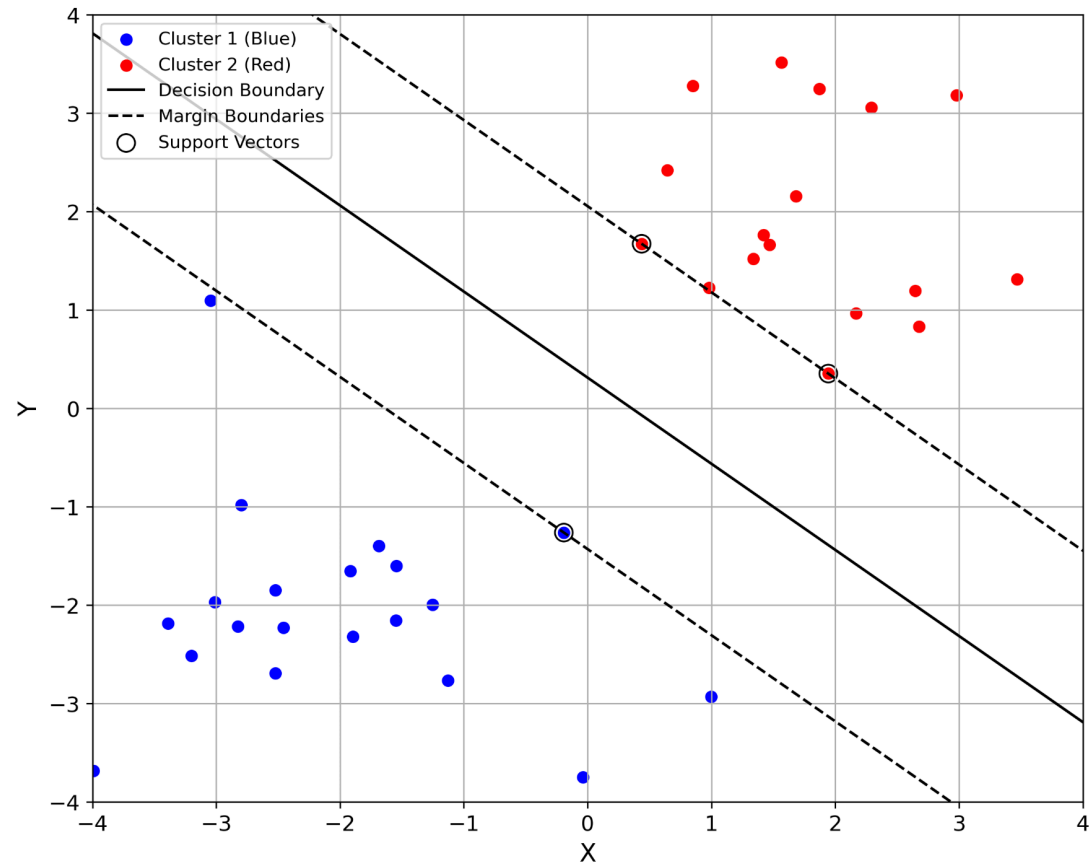
Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

However, there are several hyperplanes you may choose for separating them into correct classes.

SVM: which hyperplane separates them with the largest margin?

→ Maximum Margin



Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

What is Margin?

Given a decision boundary, $w^T x + b = 0$, the SVM will have another two parallel lines:

$$w^T x + b = 1 \text{ and } w^T x + b = -1$$

The distance between margin boundaries is $\frac{2}{||w||}$.

Thus, the aim of SVM is to enlarge the margin and minimize $||w||$. So, the final objective will be

$$\min_w \frac{1}{2} ||w||^2$$

Support Vector Machine

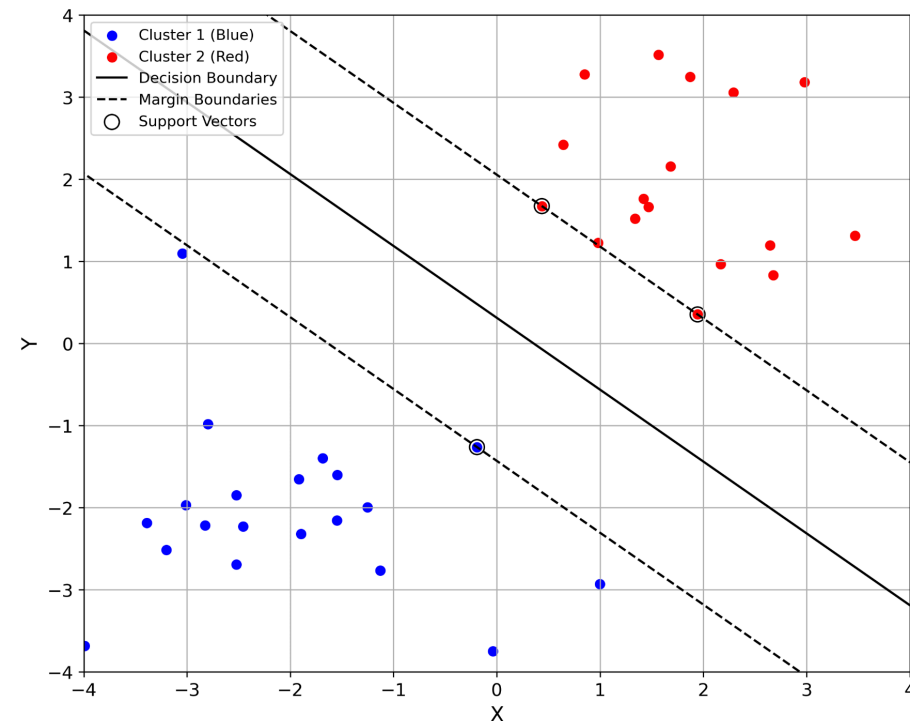
Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Why we call it as **SUPPORT VECTOR**?

It is because the decision is made by the points near by the decision boundary.

$$w^T x_i + b = +1 \text{ or } w^T x_i + b = -1$$

In a hard-margin SVM, the closest observations lying on the margin are the support vectors. In other words, the further points do not affect the hyperplane, even they move a little bit.



Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Hard-margin SVM

If our data could be divided perfectly as

$$y_i(w^T w_i + b) \geq 1$$

If $y_i = +1$, then $w^T w_i + b \geq 1$.

If $y_i = -1$, then $-(w^T w_i + b) \geq 1$.

Integrated together,

$$y_i(w^T w_i + b) \geq 1$$

Objective:

$$\min_w \frac{1}{2} ||w||^2$$

Subject to:

$$y_i(w^T w_i + b) \geq 1$$

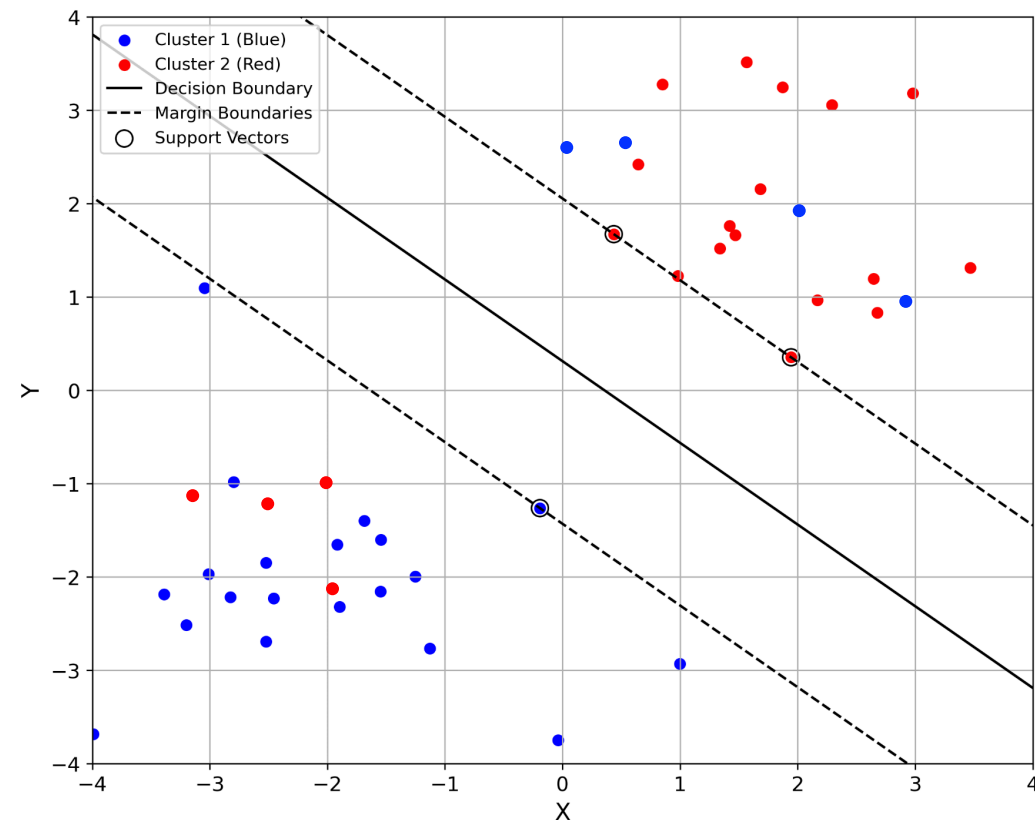
Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

In reality, we have to face few observations cannot be categorized because ...

If we still want to find a solution, then it will have no solution. Thus, SVM uses a slack variable (ξ_i) to accept some data disobey our classification.

$$y_i(w^T x_i + b) \geq 1 - \xi_i, s.t. \xi_i \geq 0$$



Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

A new objective function

$$\min_{w,b,\xi} \underbrace{\frac{1}{2} ||w||^2}_{\text{Larger margin}} + \underbrace{C \sum_i \xi_i}_{\text{Fewer classification violation}}$$

SVM = Large Margin + Classification Error Penalty

The larger C, the low incorrect classification for training data.

→ penalty↑, margin width↓, variance↑, overfitting risk↑

Larger C → less regularization; Smaller C → higher regularization

Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Soft-margin SVM

Objective function will be like

$$\min_{w,b,\xi} \frac{1}{2} ||w||^2 + C \sum_i \boxed{\max(0, 1 - y_i f(x_i))}$$

Hinge loss

where $f(x) = w^T x + b$

Let $z = y_i f(x_i)$, then $L = \max(0, 1 - z)$

$z \geq 1$	$L = 0$	Correctly classified and outside of margin
$0 < z < 1$	$L > 0$	Too close to decision boundary
$z < 0$	$L > 0$	Incorrectly classified

For SVM:

Classified correctly with sufficient confidence/
margin

Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

How to integrate objective function and constraint.

$$\min_{w,b} \frac{1}{2} \|w\|^2, s.t., y_i(w^T x_i + b) \geq 1$$

For each constraint, we apply Lagrange Multiplier ($\alpha \geq 0$).
Therefore,

$$L(w, b, \alpha) = \frac{1}{2} \|w\|^2 - \sum_i \alpha_i [y_i(w^T x_i + b) - 1]$$

Take the derivate with respect to w

$$\frac{\partial L}{\partial w} = w - \sum_i \alpha_i y_i x_i = 0; w = \sum_i \alpha_i y_i x_i$$

α_i represents how strongly training observation i contributes to the separating hyperplane.

Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

How to determine the support vectors?

$$w = \sum_i \alpha_i y_i x_i$$

$\because \alpha_i = 0$, for most observations

$$\therefore w = \sum_{i: \alpha_i > 0} \alpha_i y_i x_i$$

$\because \alpha_i > 0 \Rightarrow$ support vector

$$\therefore w = \sum_{i \in SV} \alpha_i y_i x_i$$

Only observations with non-zero α_i directly determine the final decision boundary.

Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

What is the relationship between α and C ?

For Soft-margin SVM,

$$0 \leq \alpha_i \leq C$$

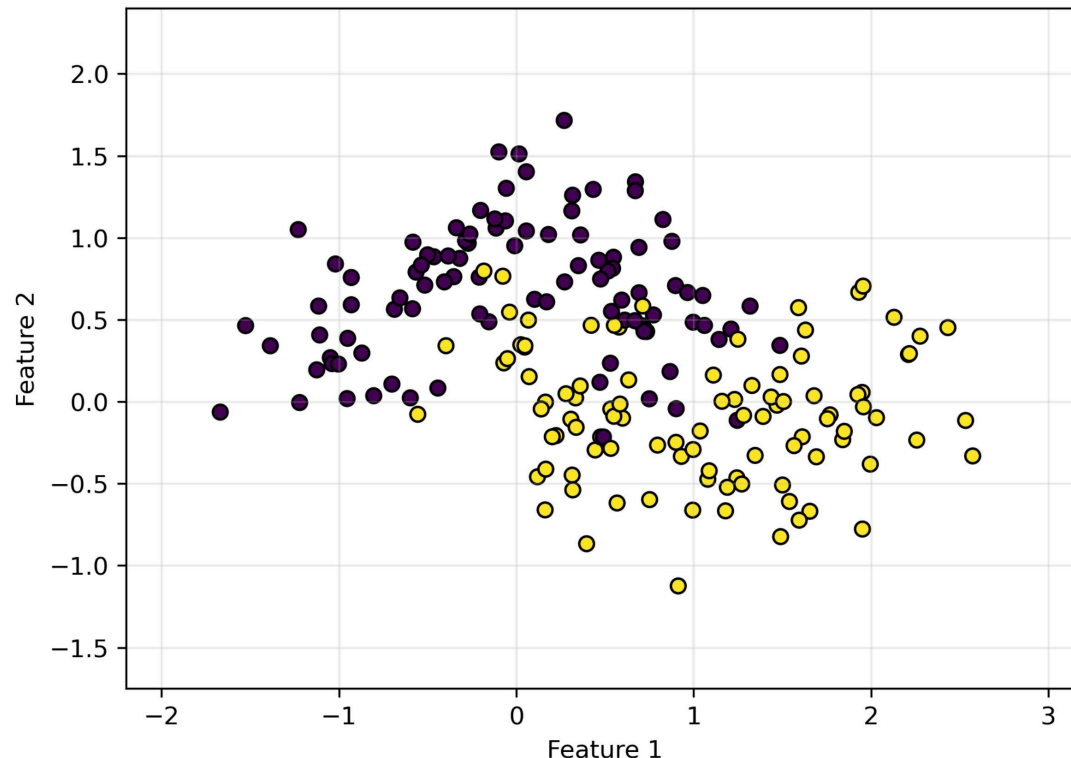
C does not directly appear in the prediction equation, but it constrains the learn α_i .

In training process, C controls the maximum weight of each observation, the predictions use the trained α_i .

Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

What if our data looks like the following figure?



We have no chance to separate them via linear boundary; therefore, we may increase the dimensionality of x to a higher one $\phi(x)$, for example, if original one is (x_1, x_2) , we may use $x_1^2 + x_2^2$, which may be linear line in high dimension, e.g., 3D.

Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

High Dimensionality Problem

If we want to solve $\phi(x)$, we may face a very high dimensional function or even infinite dimension.

In SVM, we may simplify with $\phi(x_i)^T \phi(x_j)$, thus, we may define it as $K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$, which is a kernel function.

There are three common used kernel functions.

- 1) Linear kernel
- 2) Polynomial kernel
- 3) RBF kernel

Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Why we need to use inner product in SVM?

For a **linear SVM**,

$$f(x) = w^T x + b$$

$$w = \sum_i \alpha_i y_i x_i$$

$$f(x) = \sum_i \alpha_i y_i \boxed{x_i^T x_j} + b$$

Inner Product

For a **nonlinear SVM**,

$$f(x) = \sum_i \alpha_i y_i \boxed{\phi(x_i)^T \phi(x_j)} + b$$

High Dimension
Inner Product

$$\because K(x_i, x_j) = \phi(x_i)^T \phi(x_j)$$

$$f(x) = \sum_i \alpha_i y_i K(x_i, x_j) + b$$

Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Kernel Functions

a) Linear Kernel

$$K(x_i, x_j) = x_i^T x_j$$

Suitable for high dimensional, text classification, high feature numbers

b) Polynomial Kernel

$$K(x_i, x_j) = (\gamma x_i^T x_j + r)^d$$

Able to establish polynomial decision boundary

c) RBF Kernel (Radial Basis Function/ Gaussian)

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

Nearer distance between two points, higher similarity (approx. to 1)
The gamma here affects the locality of boundary.

Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

In RBF SVM, there are two important hyperparameters. C and γ represent penalty and locality, respectively.

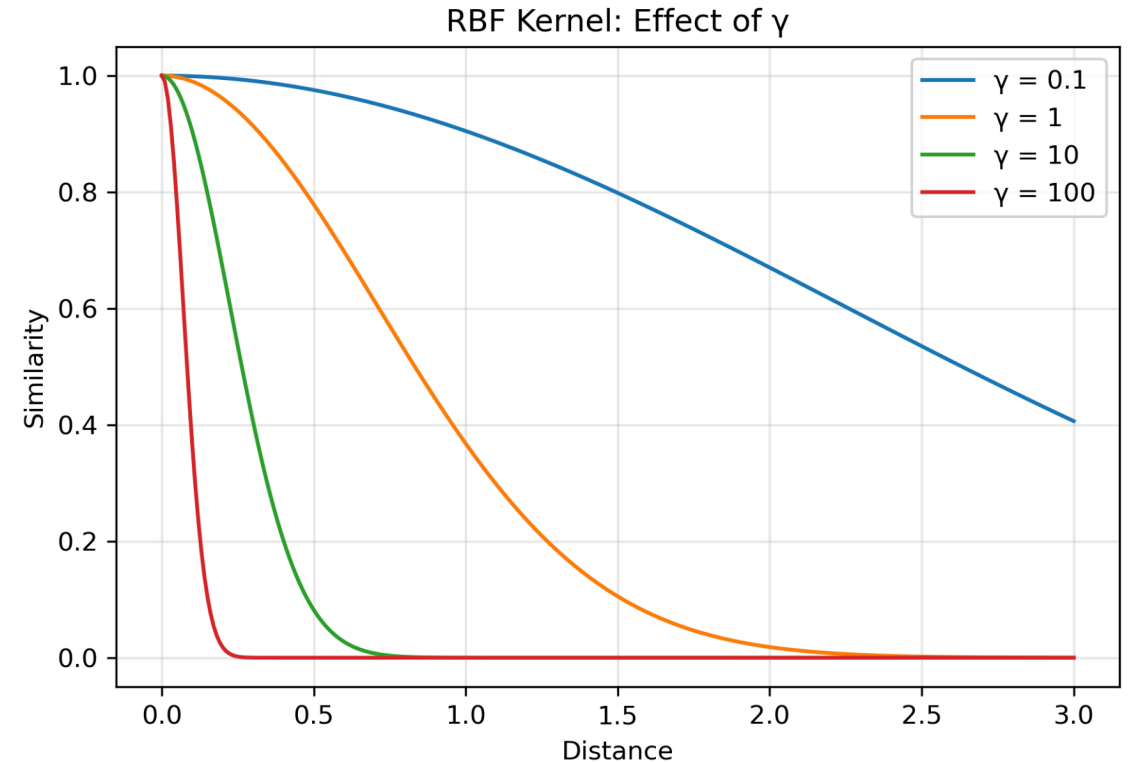
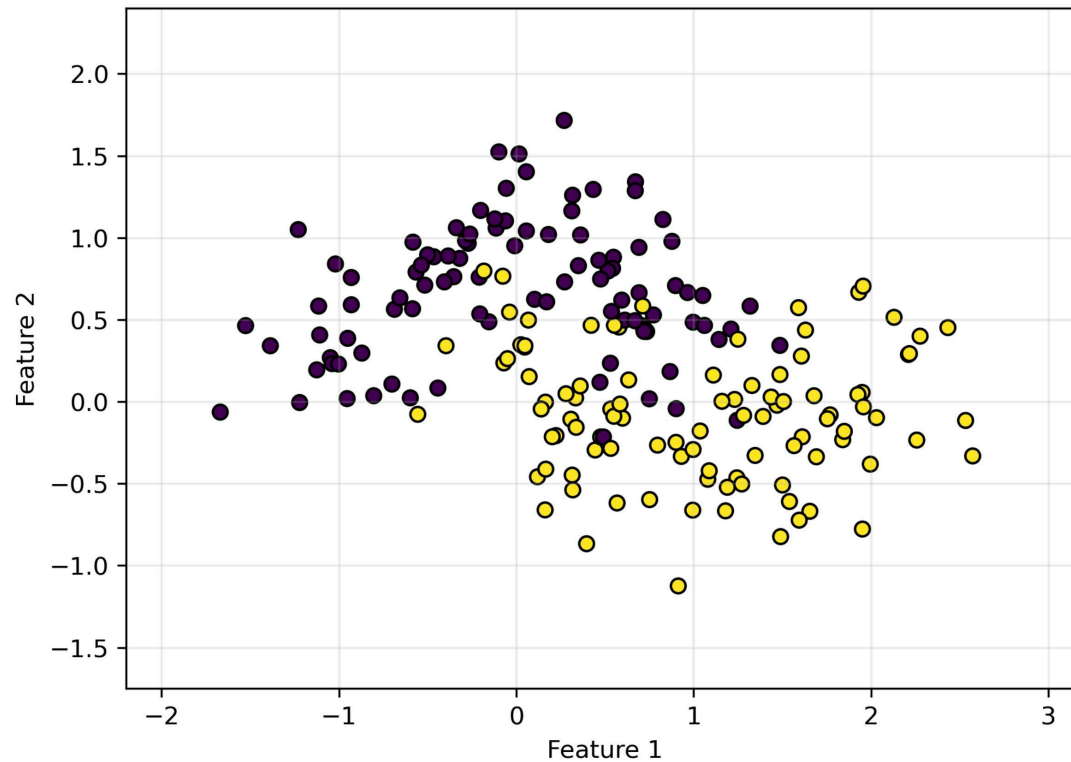
Parameter	Controls	Too Small	Too Large
C	How much classification error is tolerated? Penalty for margin violations	Under fit	May overfit
γ	How local each training point's influence is? Decay of RBF similarity with distance	Overly smooth	Overly local May overfit

	Small (γ)	Large (γ)
Small (C)	Very smooth	Moderately flexible
Large (C)	Tighter fit	Extremely flexible

Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

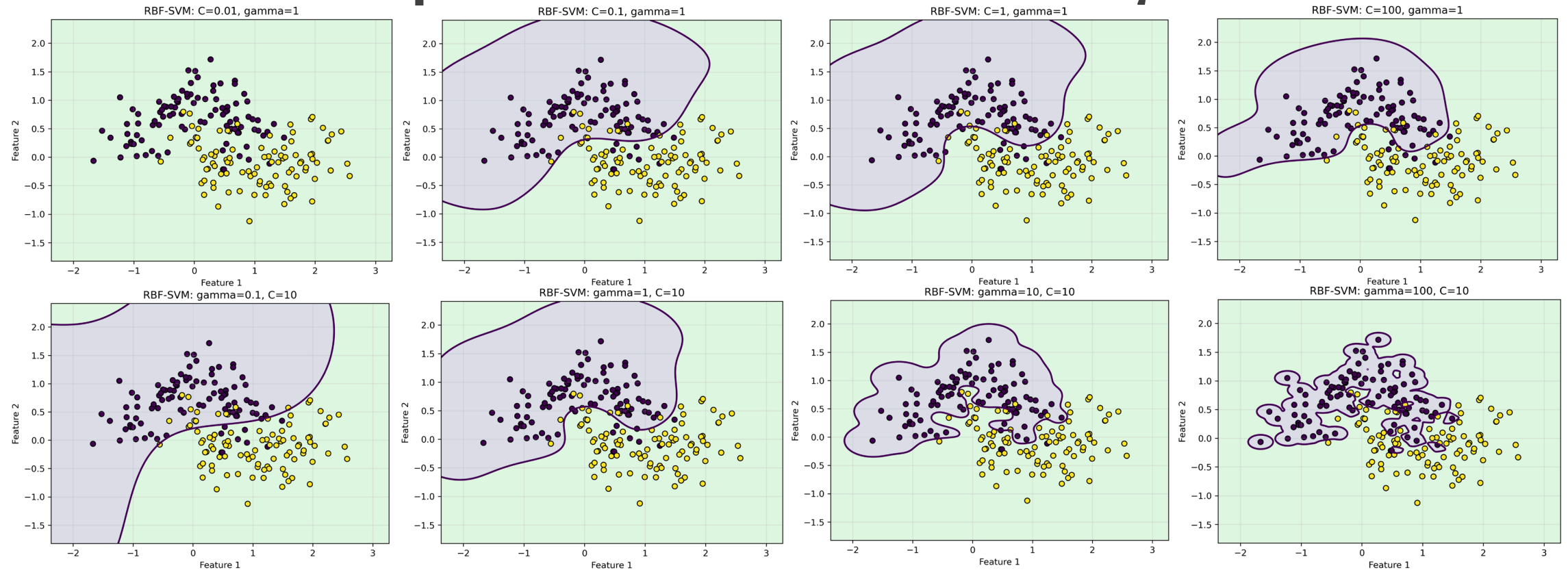
The relationship of between Outcome and γ & C in RBF



Support Vector Machine

Fundamental Classifiers | Ensemble Learning | Modern Classifiers

The relationship of between Outcome and γ & C in RBF



Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

SVM Prediction

For a **linear SVM**, we have $f(x) = w^T x + b$, thus,

$$\hat{y} = \begin{cases} +1 & f(x) > 0 \\ -1 & f(x) < 0 \end{cases}$$

For a **Kernel SVM**, we may rewrite $f(x)$ as

$$f(x) = \sum_{i \in SV} \alpha_i y_i K(x_i, x) + b, \text{ where } i \in SV$$

where SV is support vectors.

Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

A Comparison with Logistic Regression

Logistic Regression asks how probable a class is, while SVM asks how confidently the classes can be separated.

	Logistic Regression	SVM
Objective	Probability modelling	Maximum margin
Loss	Log loss	Hinge loss
Output	$P(y = 1 x)$	Decision score $f(x)$
Key observations	All samples	Mainly support vectors
Nonlinearity	Needs feature transformation	Kernel trick
Regularization	L_1/L_2	controlled by (C)
Probability	Naturally available	Not inherently probabilistic

Support Vector Machine

Fundamental Classifiers | Ensemble Learning | **Modern Classifiers**

Advantages of SVM

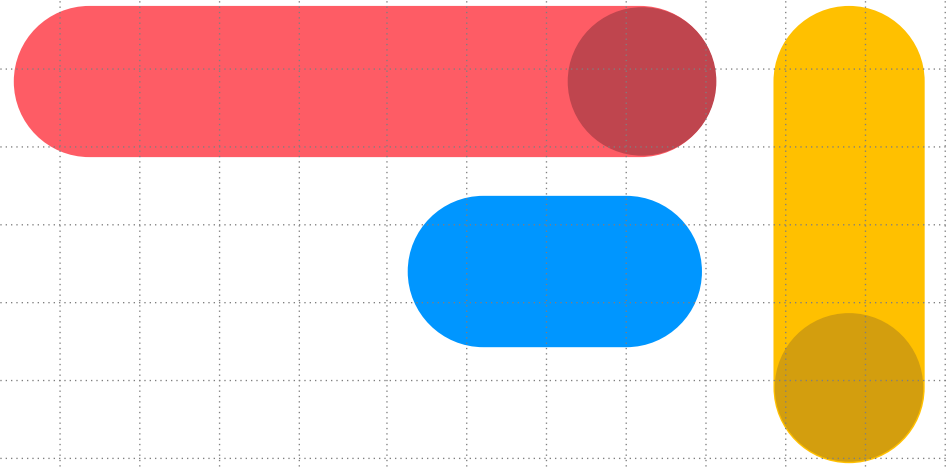
- 1) Small- to medium-sized datasets
- 2) High-dimensional features
- 3) Nonlinear classification
- 4) Cases where # of features >> # of samples

Disadvantages of SVM

- 1) SVM will calculate the distance between observations; hence, standardization is crucial for SVM.
- 2) Large dataset computationally expensive.
- 3) The original outcome of SVM is decision score, not probability

References

- Chen, T., & Guestrin, C. (2016, August). Xgboost: A scalable tree boosting system. In *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining* (pp. 785-794).
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., ... & Liu, T. Y. (2017). Lightgbm: A highly efficient gradient boosting decision tree. *Advances in neural information processing systems*, 30.
- Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: unbiased boosting with categorical features. *Advances in neural information processing systems*, 31.
- Boser, B. E., Guyon, I. M., & Vapnik, V. N. (1992). A training algorithm for optimal margin classifiers. *Proceedings of the Fifth Annual Workshop on Computational Learning Theory*, 144-152. <https://doi.org/10.1145/130385.130401>
- Cortes, C., & Vapnik, V. (1995). Support-vector networks. *Machine Learning*, 20, 273-297. <https://doi.org/10.1007/BF00994018>
- Burges, C. J. C. (1998). A tutorial on support vector machines for pattern recognition. *Data Mining and Knowledge Discovery*, 2(2), 121-167. <https://doi.org/10.1023/A:1009715923555>
- Schölkopf, B., & Smola, A. J. (2002). *Learning with kernels: Support vector machines, regularization, optimization, and beyond*. MIT Press. <https://doi.org/10.7551/mitpress/4175.001.0001>
- Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction* (2nd ed.). Springer. [Springer book page](#)
- Chang, C.-C., & Lin, C.-J. (2011). LIBSVM: A library for support vector machines. *ACM Transactions on Intelligent Systems and Technology*, 2(3), Article 27, 1-27. <https://doi.org/10.1145/1961189.1961199>
- Platt, J. C. (1999). Probabilistic outputs for support vector machines and comparisons to regularized likelihood methods. In A. J. Smola, P. L. Bartlett, B. Schölkopf, & D. Schuurmans (Eds.), *Advances in large margin classifiers* (pp. 61-74). MIT Press.



The End

Thank you for your attention!

Email: chchan@ntnu.edu.tw

Website: <https://toodou.github.io/>